

สรุปบทเรียน Object Detection Optimization

สำหรับระบบ AI ใช้งานจริง

AI in the Real World: Trade-offs Behind Fast & Scalable Object Detection

Super AI Engineer Thailand 2026 Workshop

Workshop Roadmap — 6 Labs, 3 Hours

Lab 1 Baseline Profiling FLOPs · Latency · mAP ~30 min / Google Colab	Lab 2 Structured Pruning Channel Pruning + Fine-tune ~30 min / Google Colab	Lab 3 Quantization FP32 → FP16 → INT8 ~30 min / Google Colab
Lab 4 Knowledge Distillation Teacher → Student ~30 min / Google Colab	Lab 5 Multi-Scale Heads FPN · DW-Sep · NAS Concepts ~30 min / Google Colab	Lab 6 Deployment & Scale ONNX · Multi-Camera · Edge ~30 min / Google Colab

Tensong Pichonquien, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

2

รูปที่ 2: Workshop Roadmap 6 Labs ภาพรวมเส้นทางเรียนรู้ตั้งแต่ Profiling จนถึง Deployment

เอกสารนี้จัดทำเป็นภาษาไทยเพื่ออ่านทบทวน เตรียมสอบ และใช้ต่อยอดทำ Workshop / Project ด้าน Computer Vision

คำนำ

เอกสารนี้สรุปและเรียบเรียงจากสไลด์ SAIE 2026 เรื่องการทำ Object Detection ให้เร็ว พอดีกับฮาร์ดแวร์

และขยายไปใช้กับระบบหลายกล้องได้จริง เนื้อหาถูกจัดใหม่ให้อ่านง่ายเหมือนหนังสือเรียน โดยยังคงแกนหลักจากสไลด์ ได้แก่ Baseline Profiling, Structured Pruning, Quantization, Knowledge Distillation, Multi-Scale Heads และ Edge Deployment

เหมาะสำหรับนักเรียน นักศึกษา และผู้เริ่มทำงาน AI / Computer Vision ที่ต้องการเข้าใจว่า “โมเดลที่ดี” ไม่ได้มีแค่ accuracy สูง แต่ต้องเร็ว วัดผลได้ deploy ได้ และตอบโจทย์ข้อจำกัดของโลกจริง เช่น จำนวนกล้อง FPS RAM พลังงาน และ GPU ที่มีจำกัด

สารบัญ

1. บทที่ 1 ภาพรวม Object Detection ในโลกจริง
2. บทที่ 2 คำศัพท์พื้นฐานที่ต้องรู้
3. บทที่ 3 Dataset: VisDrone
4. บทที่ 4 Metrics และสูตรที่ต้องจำ
5. บทที่ 5 Baseline Profiling และ Pareto Frontier
6. บทที่ 6 Structured Pruning
7. บทที่ 7 Quantization: FP32, FP16, INT8
8. บทที่ 8 Knowledge Distillation
9. บทที่ 9 Multi-Scale Detection และ FPN
10. บทที่ 10 Lightweight Detection Head
11. บทที่ 11 Multi-Camera Scalability และ Edge Deployment
12. บทที่ 12 ตารางเปรียบเทียบเทคนิค Optimization
13. บทที่ 13 Final Challenge / Case Study
14. บทที่ 14 Key Takeaways
15. บทที่ 15 Glossary คำศัพท์สำคัญ
16. บทที่ 16 แบบฝึกหัดท้ายบทพร้อมเฉลย
17. บทที่ 17 สรุปสุดท้ายก่อนสอบ

บทที่ 1: ภาพรวม Object Detection ในโลกจริง

Why Does This Matter? — The Real-World Problem

The Challenge from Industry

A real company needs object detection that can:

- Run fast — low latency per frame
- Handle many cameras simultaneously — without lag
- Run on embedded boards like Jetson Nano — minimal power
- Keep accuracy high enough to be trustworthy

4 Cameras × 25 FPS

= 100 FPS total
needed
from a single GPU

Embedded Hardware

Jetson Nano: only
4GB RAM, 128 CUDA
cores

mAP Must Hold

Accuracy ≥ 80%
of the baseline model

The Engineering Philosophy Behind This Workshop

Every optimization must be measurable.

We never say "it's faster" without showing numbers.

- We track: mAP@50, FLOPs, Latency (ms), Throughput (FPS), Parameters (M)

Teerapong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

3

รูปที่ 3: โจทย์อุตสาหกรรม: 4 กล้อง × 25 FPS = ต้องรองรับ 100 FPS บน GPU เดียว พร้อมรักษา mAP อย่างน้อย 80% ของ baseline

Object Detection คืออะไร

Object Detection คือการให้โมเดลมองภาพแล้วตอบว่า “มีวัตถุอะไรบ้าง อยู่ตรงไหน และมั่นใจแค่ไหน” ผลลัพธ์สำคัญคือ class label, bounding box และ confidence score ตัวอย่างเช่น ภาพจากโดรนอาจมี pedestrian, car, bus และ motor หลายตำแหน่งในภาพเดียว

ทำไมต้อง Optimize โมเดล

ในงานจริง โมเดลที่แม่นยำที่สุดอาจไม่ใช่โมเดลที่ใช้งานได้ดีที่สุด เพราะระบบจริงต้องประมวลผลภาพจำนวนมากต่อวินาที ใช้พลังงานต่ำ และทำงานบนอุปกรณ์จำกัด เช่น Jetson Nano ที่มี RAM 4GB และ CUDA cores จำกัด ดังนั้นการ optimize คือการหาจุดสมดุลระหว่างความแม่นยำ ความเร็ว หน่วยความจำ และความเสถียร

- ต้องมี latency ต่ำ เพื่อไม่ให้ภาพวิดีโอดีเลย์
- ต้องมี throughput สูง เพื่อรองรับหลายกล้องพร้อมกัน
- ต้องรักษา mAP ให้อยู่ในระดับที่เชื่อถือได้
- ต้องวัดผลทุกครั้ง ไม่พูดว่าเร็วขึ้นโดยไม่มีตัวเลข

แนวคิดหลักของ Workshop

ปรัชญาสำคัญคือ “Every optimization must be measurable” หรือทุกการปรับแต่งต้องวัดได้จริง ตัวชี้วัดที่ใช้ตลอดบทเรียนคือ mAP@50, FLOPs, latency, FPS และจำนวน parameters

บทที่ 2: คำศัพท์พื้นฐานที่ต้องรู้ก่อน

คำศัพท์	ความหมาย	สำคัญอย่างไร / ตัวอย่าง
Object Detection	การตรวจจับวัตถุในภาพ พร้อมบอกตำแหน่งด้วย bounding box	ใช้กับกล้องวงจรปิด โดรน รถยนต์อัตโนมัติ
mAP@50	Mean Average Precision ที่ IoU = 0.5	ใช้วัดความแม่นยำของการตรวจจับแบบไม่เข้มงวดมาก
mAP@50-95	ค่า mAP เฉลี่ยหลาย threshold ตั้งแต่ 0.50 ถึง 0.95	เข้มงวดกว่า เพราะลงโทษ box ที่ไม่แม่นยำตำแหน่ง
FLOPs	จำนวนการคำนวณ floating point ใน forward pass	ยิ่งน้อยยิ่งเร็วและกินพลังงานน้อย
Latency	เวลาที่ใช้ประมวลผลภาพหนึ่งภาพ	สำคัญกับระบบ real-time
FPS	จำนวน frame ที่ประมวลผลได้ต่อวินาที	ต้อง ≥ 25 FPS ต่อกล้องสำหรับ real-time
Throughput	ปริมาณภาพรวมที่ระบบประมวลผลได้ต่อวินาที	สำคัญเมื่อมีหลายกล้อง
Parameters	จำนวนค่าน้ำหนักในโมเดล	บอกขนาดโมเดลและหน่วยความจำโดยประมาณ
Efficiency Score	mAP@50 / GFLOPs	ใช้เปรียบเทียบความคุ้มค่าระหว่าง accuracy กับ compute
Edge AI	การรัน AI บนอุปกรณ์ปลายทาง	เช่น Jetson Nano กล้องอัจฉริยะ หรืออุปกรณ์ฝังตัว
ONNX	รูปแบบกลางสำหรับ export โมเดล	ช่วยย้ายโมเดลจาก PyTorch ไป runtime อื่น
TensorRT	เครื่องมือ optimize inference ของ NVIDIA	ใช้เร่งความเร็วบน GPU/Jetson

บทที่ 3: Dataset ที่ใช้ในไฟล์ - VisDrone

Dataset — VisDrone (SAIE 2026 Subset)

Setup

What is VisDrone?

VisDrone is a large-scale benchmark dataset captured by drone cameras at varying altitudes, lighting, and weather conditions.

10 object classes for urban traffic monitoring:

- pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus, motor
- Key challenge: Severe class imbalance (pedestrian >> bus)**
- Objects are tiny — many < 32x32 pixels in drone footage

~800
Train Images

~200
Val Images

10
Object Classes

416x416
Input Size

5,000
Pretrained Epochs

YOLOv8s
Model

Why VisDrone for This Workshop?

Tiny objects from drone altitude stress-test every optimization. A pruned model that loses accuracy on pedestrians can have real safety consequences — this dataset keeps us honest.

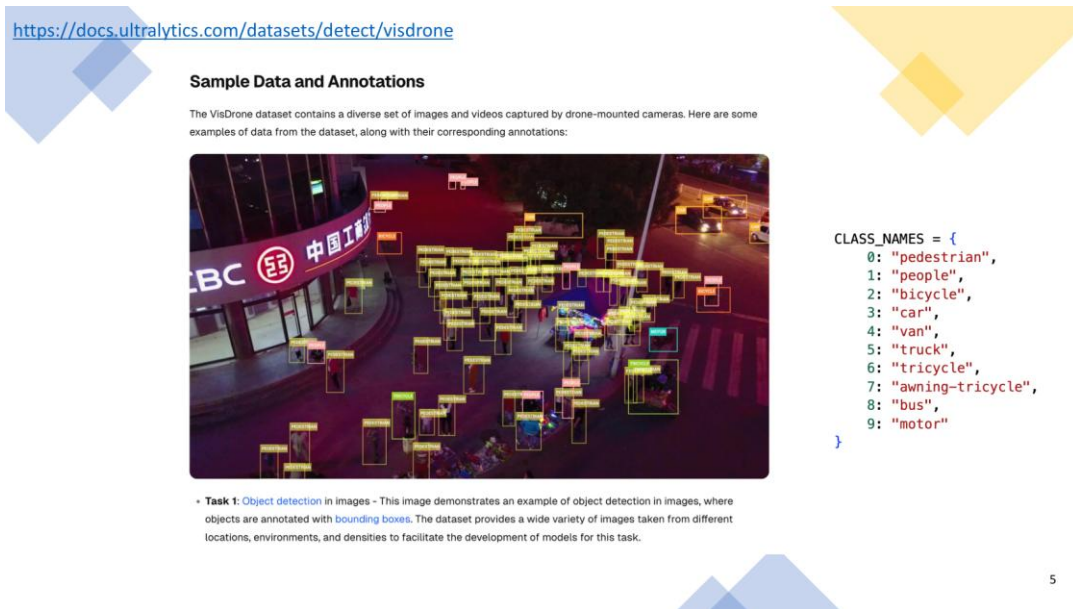
Teechai Pimboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

4

รูปที่ 4: ข้อมูล VisDrone SAIE 2026 Subset: ประมาณ 800 train images, 200 validation images, 10 classes, input 416x416, YOLOv8s baseline

VisDrone คือ dataset benchmark สำหรับงานภาพจากโดรน ซึ่งมีความหลากหลายด้านมุมมอง สภาพแสง สภาพอากาศ และฉากเมือง เหมาะสำหรับการทดสอบ Object Detection ในสถานการณ์จริงที่วัตถุจำนวนมากมีขนาดเล็กและกระจายอยู่ทั่วภาพ

Class ที่ใช้ในงาน urban traffic monitoring มี 10 class ได้แก่ pedestrian, people, bicycle, car, van, truck, tricycle, awning-tricycle, bus และ motor



รูปที่ 5: ตัวอย่างภาพ VisDrone พร้อม bounding boxes จำนวนมาก แสดงความยากของ tiny objects และ dense objects

ทำไม VisDrone ถึงยาก

- วัตถุจำนวนมากมีขนาดเล็กมาก เช่น น้อยกว่า 32x32 pixels
- มี class imbalance เช่น pedestrian เยอะกว่า bus มาก
- ภาพจากโดรนมีมุมมองสูง ทำให้วัตถุถูกบังหรือมีรายละเอียดน้อย
- การ optimize ที่ลด capacity ของโมเดลมากเกินไปอาจทำให้ตรวจจับคนเดินถนนพลาด ซึ่งมีผลต่อความปลอดภัย

บทที่ 4: Metrics และสูตรที่ต้องจำ

Key Metrics We Track — Definitions & Equations

Fundamentals

mAP@50 Mean Average Precision at IoU=0.5 $mAP = (1/ C) \times \sum AP_c$ Average of per-class Average Precision across all C classes. Primary accuracy metric for object detection.	mAP@50-95 Strict mAP (averaged over IoU 0.5→0.95) $mAP_{50:95} = (1/10) \times \sum mAP_{IoU}$ Averages over 10 IoU thresholds. Penalizes imprecise boxes. Used in COCO benchmark.	FLOPs Floating Point Operations $FLOPs = 2 \times C_{in} \times k^2 \times C_{out} \times H \times W$ Total multiply-adds in a forward pass. Measures computational cost. Lower = faster model.
Latency Inference Time per Image (ms) $Latency = t_{end} - t_{start} \text{ (ms)}$ Wall-clock time for one inference. Measure after GPU warm-up! Use P95 for worst-case.	FPS Frames Per Second (Throughput) $FPS = 1000 / \text{mean_latency_ms}$ How many frames per second the model can process. Must be ≥ 25 FPS per camera for real-time.	Efficiency Accuracy-per-Compute Trade-off $Efficiency = mAP_{50} / GFLOPs$ Measures how much accuracy you get per unit of compute. Higher = better. Helps compare models fairly.

Teerapong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

7

รูปที่ 7: สรุป metrics หลักที่ใช้วัดโมเดล: mAP@50, mAP@50-95, FLOPs, Latency, FPS และ Efficiency

Precision

$$Precision(k) = TP(k) / (TP(k) + FP(k))$$

บอกว่า detection ที่โมเดลทำนายว่าเจอวัตถุนั้น ถูกจริงกี่ส่วน ยิ่งสูงยิ่ง false positive น้อย

Recall

$$Recall(k) = TP(k) / \text{Total Ground Truth}$$

บอกว่าในวัตถุจริงทั้งหมด โมเดลหาเจอมากแค่ไหน ยิ่งสูงยิ่งพลาดน้อย

Average Precision (AP)

$$AP_c = \int P(r) dr \approx \sum P(k) \times \Delta R(k)$$

พื้นที่ใต้กราฟ Precision-Recall ของแต่ละ class

mAP

$$mAP = (1 / |C|) \times \sum AP_c$$

ค่าเฉลี่ย AP ทุก class ใช้เป็น metric หลักของ object detection

FLOPs Conv2D

$$FLOPs = 2 \times C_{in} \times k^2 \times C_{out} \times H \times W$$

จำนวน multiply-add ของ convolution หนึ่ง layer

FPS

$$\text{FPS} = 1000 / \text{mean_latency_ms}$$

แปลงเวลาหน่วยเฉลี่ยต่อภาพเป็นจำนวนภาพต่อวินาที

Efficiency Score

$$\text{Efficiency} = \text{mAP@50} / \text{GFLOPs}$$

วัดความคุ้มของ accuracy ต่อ compute

Mathematical Foundation

$$\text{mAP} = \frac{1}{|C|} \sum_{c \in C} \text{AP}_c$$

$$\text{AP}_c = \int_0^1 p(r) dr \approx \sum_{k=1}^N P(k) \cdot \Delta R(k)$$

Where $P(k)$ is precision at rank k , and $\Delta R(k)$ is the change in recall.

$$\text{FPS} = \frac{1}{\text{Average Latency per Frame (s)}}$$

$$\text{Efficiency Score} = \frac{\text{mAP}}{\text{GFLOPs}} \quad (\text{higher} = \text{better})$$

25

รูปที่ 25: Mathematical Foundation: สูตร mAP, AP, FPS และ Efficiency Score ที่ควรจำก่อนสอบ

ข้อควรระวังของ metrics

- mAP@50 สูงไม่ได้แปลว่า box แม่นมาก เพราะ threshold ผ่อนกว่า mAP@50-95
- Mean latency อย่างเดียวไม่พอ ควรดู P95 latency เพื่อเห็น worst-case lag
- FLOPs ต่ำไม่ได้แปลว่าเร็วเสมอ เพราะขึ้นกับ hardware, batch size และ runtime
- FPS ต่อกลึงกับ total throughput เป็นคนละเรื่อง ต้องคิดให้ชัดเมื่อมีหลายกลึง

บทที่ 5: Baseline Profiling และ Pareto Frontier

ก่อน optimize ต้องรู้ baseline ก่อนว่าโมเดลเดิมมีความแม่นยำ ความเร็ว และขนาดเท่าไร ถ้าไม่วัด baseline จะไม่รู้ว่าการ prune, quantize หรือ distill ช่วยจริงหรือแค่รู้สึกว่าร็วขึ้น

สิ่งที่ต้องวัดใน Baseline

- mAP@50 และ mAP@50-95 เพื่อดู accuracy
- Latency แบบ mean, P50 และ P95 เพื่อดูเวลาหน่วงจริง
- FPS เพื่อดู throughput
- FLOPs เพื่อดูต้นทุน compute
- Parameters เพื่อดูขนาดโมเดล

ทำไมต้อง warm up GPU และ torch.cuda.synchronize()

GPU ช่วงแรกอาจช้ากว่าปกติจาก CUDA kernel compilation และ memory allocation จึงต้อง warm up ก่อนวัดจริง นอกจากนี้ GPU ทำงานแบบ asynchronous ถ้าไม่เรียก torch.cuda.synchronize() อาจวัดได้แค่เวลาสั่งงาน ไม่ใช่เวลาคำนวณจริง

Pareto Frontier

Pareto Frontier คือแนวคิดที่บางครั้งเราไม่สามารถเพิ่ม metric หนึ่งโดยไม่เสียอีก metric หนึ่ง เช่น ถ้าต้องการ mAP สูงขึ้น อาจต้องใช้โมเดลใหญ่ขึ้นและช้าลง ถ้าต้องการเร็วขึ้น อาจต้องยอมลด accuracy บางส่วน ดังนั้นโมเดลที่เหมาะสมที่สุดไม่ใช่โมเดลที่ “ดีที่สุดทุกด้าน” แต่เป็นโมเดลที่อยู่ในจุดคุ้มค่าตาม constraint

Model	Params	GFLOPs	FPS (T4)	mAP@50	การตีความ
YOLOv8n	3.2M	8.7G	160	0.37	เร็วที่สุด เหมาะกับ edge / multi-camera
YOLOv8s	11.2M	28.6G	120	0.45	baseline ที่สมดุลสำหรับ workshop
YOLOv8m	25.9M	78.9G	80	0.50	accuracy สูงขึ้น แต่ช้าลง
YOLOv8l	43.7M	165G	50	0.53	แม่นยำแต่ไม่เหมาะกับ edge multi-camera

บทที่ 6: Structured Pruning

Pruning คือการตัดส่วนที่ไม่ค่อยมีผลต่อ output ออกจากโมเดล เพื่อลดขนาดและลดการคำนวณ ในสไลด์เน้น Structured Pruning ซึ่งตัดเป็น channel หรือ filter ทั้งชุด ทำให้โมเดลเล็กลงจริงและเร็วขึ้นจริงบน GPU

Structured vs Unstructured Pruning

ประเภท	แนวคิด	ผลต่อความเร็วจริง
Unstructured Pruning	ตั้งค่าน้ำหนักบางตัวเป็นศูนย์	อาจไม่ลด FLOPs บน GPU เพราะโครงสร้าง tensor ยังเหมือนเดิม

Structured Pruning	ลบ channel/filter ทั้งชุดออกจากโมเดล	ลด FLOPs และ latency ได้จริงกว่า
--------------------	--------------------------------------	----------------------------------

Channel Importance ด้วย L1-Norm

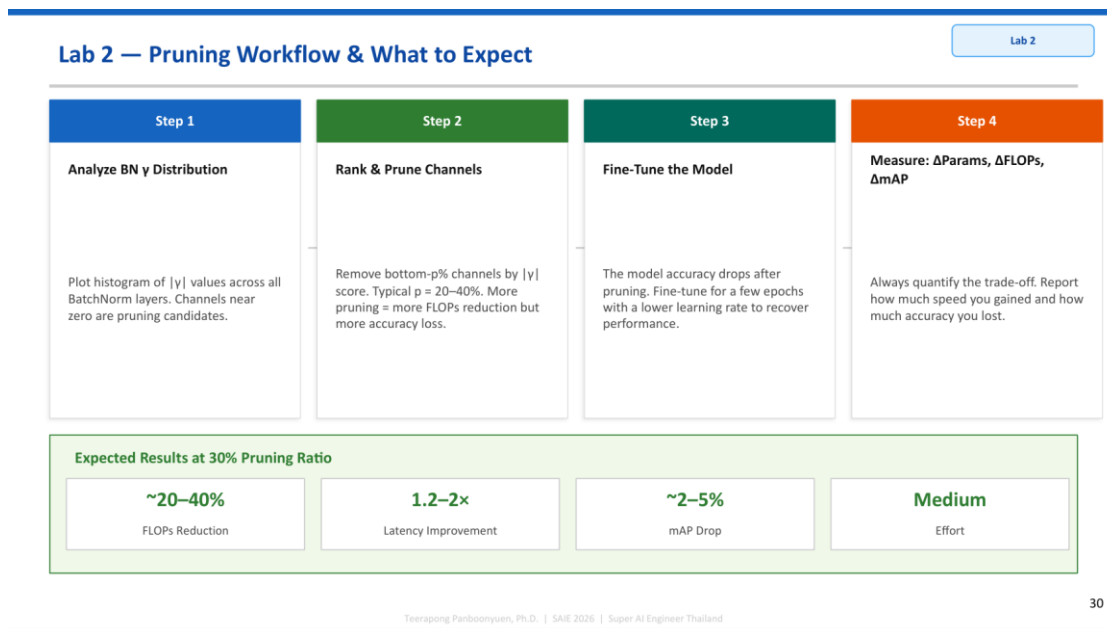
$$s_i = \|F_i\|_1 = \sum |F_i(j,k,l)|$$

ถ้า filter มีค่า L1-Norm ต่ำ แปลว่า activation หรือ contribution ต่อ output อาจต่ำ จึงเป็น candidate สำหรับการ prune

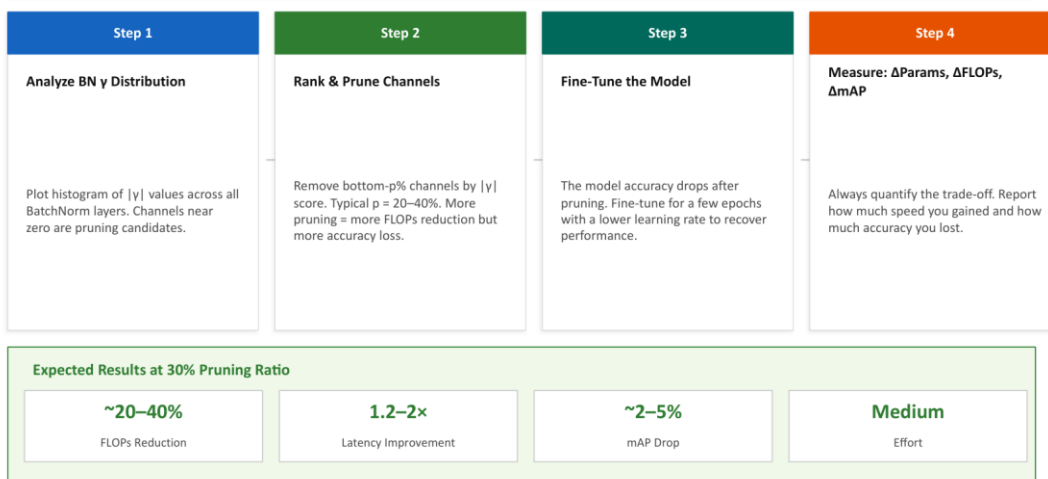
BatchNorm Gamma Pruning

$$y = \gamma \times \hat{x} + \beta, \quad \hat{x} = (x - \mu) / \sqrt{\sigma^2 + \epsilon}$$

ใน BatchNorm ค่า gamma (γ) ทำหน้าที่เหมือนตัวปรับ scale ของ channel ถ้า $|\gamma|$ ใกล้ศูนย์ output ของ channel นั้นจะเล็กมาก จึงเป็นสัญญาณว่า channel นั้นอาจตัดได้



Lab 2 — Pruning Workflow & What to Expect



30

Teerapong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

รูปที่ 30: Workflow ของ Structured Pruning: วิเคราะห์ BN gamma, prune, fine-tune และวัดผลหลังปรับ

ผลที่คาดหวังเมื่อ prune 30%

- FLOPs ลดประมาณ 20-40%
- Latency ดีขึ้นประมาณ 1.2-2 เท่า
- mAP อาจลดประมาณ 2-5%
- ต้อง fine-tune หลัง prune เพื่อกู้ accuracy กลับมา

บทที่ 7: Quantization - FP32, FP16, INT8

Quantization คือการลดจำนวน bit ที่ใช้แทนค่าน้ำหนักหรือ activation เช่น จาก FP32 เป็น FP16 หรือ INT8

จุดประสงค์คือให้โมเดลเล็กลง ใช้ memory bandwidth น้อยลง และ inference เร็วขึ้น

Uniform Affine Quantization

$$s = (x_{\max} - x_{\min}) / (2^b - 1)$$

$$z = -\text{round}(x_{\min} / s)$$

$$q = \text{clamp}(\text{round}(x/s) + z, 0, 2^b - 1)$$

$$x_{\text{hat}} = s \times (q - z), \quad |x - x_{\text{hat}}| \leq s/2$$

เปรียบเทียบ precision

Precision	ขนาดเทียบกับ FP32	จุดเด่น	ข้อควรระวัง
FP32	1.0x	แม่นยำและเสถียร ใช้เป็น baseline	ช้าและใช้ memory มากกว่า
FP16	0.5x	มักเร็วขึ้น 1.5-2x และ accuracy แทบไม่ลด	ต้องมี hardware รองรับ FP16
INT8	0.25x	ประหยัด memory 4x และเร็ว 2-4x บน hardware ที่รองรับ	ต้องใช้ calibration dataset และอาจ mAP ลด 1-2%
INT4	0.125x	เล็กมาก	ความแม่นยำเสี่ยงลดมากกว่าใช้ยากกว่า

Lab 3 — FP16 vs INT8: When to Use Each

Lab 3

FP16 (Half Precision)	INT8 (8-bit Integer)	ONNX Runtime
✓ Advantages: + Zero accuracy loss on modern GPUs + Enabled with one flag (half=True) + Works on all CUDA-capable hardware + ~1.5–2× faster on T4, A100, RTX ✗ Limitations: - No benefit on hardware without FP16 units - Slight memory overhead vs INT8 Recommendation: Use FP16 always as a first optimization. It's free speed.	✓ Advantages: + 4× memory savings vs FP32 + 2–4× faster on dedicated INT8 hardware + Ideal for Jetson Nano TensorRT + Enables running 4+ cameras on edge ✗ Limitations: - Requires calibration dataset - ~1–2% mAP drop - Needs TensorRT or ONNX runtime Recommendation: Use INT8 for edge deployment (Jetson Nano) after FP16.	✓ Advantages: + Hardware-agnostic inference + Works on CPU, GPU, edge + Easy export from PyTorch/YOLO + Enables cross-platform deployment ✗ Limitations: - Slightly slower than TensorRT - Less tuned for specific hardware - No training — inference only Recommendation: Use ONNX as a step toward TensorRT on edge devices.

33

Teerapong Panboonyom, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

รูปที่ 33: สไลด์เปรียบเทียบ FP16, INT8 และ ONNX Runtime ว่าควรใช้เมื่อใด

คำแนะนำจากสไลด์

- เริ่มจาก FP16 ก่อน เพราะเป็น free speed ใน GPU สมัยใหม่
- ใช้ INT8 สำหรับ edge deployment เช่น Jetson Nano + TensorRT
- ใช้ ONNX เป็นสะพานกลางก่อนแปลงไป runtime ที่เหมาะกับ hardware

บทที่ 8: Knowledge Distillation

Knowledge Distillation คือการให้โมเดลใหญ่ที่เก่งกว่า (Teacher) ถ่ายทอดความรู้ให้โมเดลเล็กกว่า (Student) เพื่อให้ student ทำงานเร็วเท่าโมเดลเล็ก แต่ได้ accuracy ใกล้ teacher มากขึ้น

Hard Label vs Soft Label

ประเภท Label	ตัวอย่าง	ความหมาย
Hard Label	car: 1.0, van: 0.0, truck: 0.0	บอกแค่ว่าคำตอบถูกคืออะไร
Soft Label	car: 0.80, van: 0.15, truck: 0.04	บอกความคล้ายระหว่าง class ด้วย เช่น car คล้าย van มากกว่า bus

Dark Knowledge คือข้อมูลความสัมพันธ์ระหว่าง class ที่อยู่ใน output probability ของ teacher แม้ class รองจะไม่ใช่คำตอบจริง แต่ช่วยให้ student generalize ได้ดีขึ้น

Temperature Scaling และ KD Loss

$$p_i^T = \exp(z_i/T) / \sum_j \exp(z_j/T)$$

$$L_{KD} = (1 - \alpha) L_{CE}(\text{hard}) + \alpha T^2 L_{KL}(\text{teacher} \parallel \text{student})$$

- T สูงขึ้นทำให้ distribution นุ่มขึ้น เห็น dark knowledge ชัดขึ้น
- α คือสัดส่วนความสำคัญของ distillation loss
- T^2 ช่วยชดเชย scale ของ gradient เมื่อใช้ temperature

ทำไมเหมาะกับระบบหลายกล้อง

เมื่อต้องรัน inference จำนวนมากต่อวินาที student ที่เร็วกว่า teacher มากจะช่วยรองรับหลายกล้องได้ดีขึ้น สไลด์ยกตัวอย่างว่า YOLOv8n student ที่ฝึกด้วย KD อาจได้ mAP ประมาณ 92% ของ YOLOv8s teacher แต่เร็วขึ้นประมาณ 2.5 เท่า

Final Hackathon Challenge — Design for 4 Cameras at 25 FPS

Mission: Design a system to run 4 cameras at 25 FPS each using one GPU (T4 or Jetson Nano), keeping mAP@50 ≥ 80% of baseline.

Option	Est. Total FPS	Meets 100 FPS?	mAP ≥ 80%?
YOLOv8s FP32	~120	✗	✓
YOLOv8s FP16	~200	✓	✓
YOLOv8n FP16	~300	✓	✓ (~87%)
YOLOv8n FP16 + KD	~300+	✓	✓ (~92%)
YOLOv8n INT8 (Jetson)	~25/cam	✓	✓ (~89%)

Best Recommendation: YOLOv8n FP16 + Knowledge Distillation

- Highest FPS while keeping mAP within 8% of baseline
- With batch inference (bs=4): throughput increases ~2× further → exceeds 600 img/s
- For Jetson Nano: YOLOv8n INT8 gives ~20-25 FPS per camera — adequate for 1-2 cameras, scale to 4 with a stronger board

Teerapong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

44

รูปที่ 44: Challenge และ recommendation: YOLOv8n FP16 + Knowledge Distillation เป็นตัวเลือกที่คุ้มสำหรับ 4 กล้อง

บทที่ 9: Multi-Scale Detection และ Feature Pyramid Network

Lab 5 — Feature Pyramid Network (FPN) & Multi-Scale Detection

Lab 5

Why Do We Need Multiple Detection Scales?

A pedestrian at drone altitude may be only 8×8 pixels. A single detection head tuned for large objects will miss it entirely.

P3 (Stride 8)	P4 (Stride 16)	P5 (Stride 32)
Grid: 52×52 Anchor points: 2,704 Best for: Tiny objects < 16 px Equation: Grid = $\lfloor 416 / 8 \rfloor \times \lfloor 416 / 8 \rfloor$	Grid: 26×26 Anchor points: 676 Best for: Small objects 16–64 px Equation: Grid = $\lfloor 416 / 16 \rfloor \times \lfloor 416 / 16 \rfloor$	Grid: 13×13 Anchor points: 169 Best for: Large objects > 64 px Equation: Grid = $\lfloor 416 / 32 \rfloor \times \lfloor 416 / 32 \rfloor$

YOLOv8 — Anchor-Free Detection (DFL)

YOLOv8 uses Distribution Focal Loss (DFL): $\hat{b} = \sum_i \text{softmax}(z_i) \times i$
 The model learns a distribution over offsets rather than fixed anchor shapes → better for diverse small objects in drone footage.

Teerapong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

38

รูปที่ 38: FPN และ multi-scale detection: P3 สำหรับ tiny objects, P4 สำหรับ small objects, P5 สำหรับ large objects

ทำไมต้องมีหลาย scale

ในภาพจากโดรน คนเดินถนนอาจมีขนาดแค่ 8×8 pixels ถ้าใช้ detection head ที่เหมาะกับวัตถุใหญ่เพียงอย่างเดียว โมเดลจะพลาด tiny objects ได้ง่าย ดังนั้นต้องมี feature pyramid ที่มองหลายระดับความละเอียด

ระดับ	Stride	Grid เมื่อ input 416×416	เหมาะกับ
-------	--------	--------------------------	----------

P3	8	52x52 = 2,704 points	Tiny objects < 16 px
P4	16	26x26 = 676 points	Small objects 16-64 px
P5	32	13x13 = 169 points	Large objects > 64 px

YOLOv8 Anchor-Free Detection และ DFL

YOLOv8 ใช้แนวคิด anchor-free และ Distribution Focal Loss (DFL) โดยให้โมเดลเรียน distribution ของ offset แทนการยึด fixed anchor shape ทำให้ยืดหยุ่นกับวัตถุหลายขนาดในภาพโดรนมากขึ้น

$$\mathbf{b_hat} = \sum \text{softmax}(z_i) \times i$$

บทที่ 10: Lightweight Detection Head

แนวคิดสำคัญคือเปลี่ยน convolution ใน detection head ให้เบาลงโดยใช้ Depthwise-Separable Convolution ซึ่งแยกการกรองเชิงพื้นที่ออกจากการผสม channel

Standard Conv2D

$$\text{FLOPs} = C_in \times k^2 \times C_out \times H \times W$$

Standard Conv จะใช้ filter ที่ผสมทั้ง spatial และ channel พร้อมกัน จึงคำนวณเยอะเมื่อ C_in และ C_out สูง

Depthwise-Separable Conv

$$\text{FLOPs}_{\text{DW}} = (C_in \times k^2 \times H \times W) + (C_in \times C_out \times H \times W)$$

$$\text{Reduction} \approx (1 / C_out) + (1 / k^2)$$

สำหรับ $k=3$ และ $C_out=256$ สไลด์ระบุว่าสามารถลด FLOPs ได้ประมาณ 8-9 เท่าในส่วน head เพราะขั้นแรกใช้ depthwise conv ต่อ channel แยกกัน แล้วขั้นสองใช้ pointwise 1×1 conv เพื่อผสม channel

สำหรับ VisDrone การลด FLOPs ที่ P3 head สำคัญมาก เพราะ P3 เป็นจุดที่ tiny pedestrians ปรากฏมากที่สุด การออกแบบ head ให้เบาแต่ยังรักษา P3 ได้จึงเป็นแนวทางที่คุ้ม

บทที่ 11: Multi-Camera Scalability และ Edge Deployment

ระบบหลายกล้องไม่ได้ต้องการแค่มอเดลเร็ว แต่ต้องมีการจัดคิวภาพ การ batch inference และการ deploy ให้เหมาะกับ hardware

Batch Inference

Batch inference คือการนำ frame จากหลายกล้องมารวมเป็น batch เดียว แล้วให้ GPU forward pass พร้อมกัน ช่วย amortize overhead เช่น memory transfer และ kernel launch ทำให้ throughput รวมสูงขึ้น แต่ latency ต่อ batch จะเพิ่มขึ้น

Batch size	Latency / batch	FPS ต่อภาพ	Total throughput
------------	-----------------	------------	------------------

bs=1	8 ms	125	125 img/s
bs=2	10 ms	100	200 img/s
bs=4	15 ms	67	267 img/s
bs=8	25 ms	40	320 img/s

ตัวอย่าง requirement: 4 กล้อง x 25 FPS = 100 images/sec ถ้า YOLOv8n FP16 + batch size 4 ได้ประมาณ 267 img/s ก็ผ่าน requirement พร้อม headroom

Lab 6 — Edge Deployment Guide: Jetson Nano + TensorRT

Lab 6

Steps to Deploy Your Optimized Model on Jetson Nano

Step 1: Export to ONNX (in Colab)

Step 2: Convert to TensorRT (on Jetson)

Step 3: Run Inference on Jetson

Step 4: Multi-Camera Streaming

Expected Jetson Nano Performance:

Single camera: ~20-25 FPS | Dual camera: ~12-15 FPS/stream | 4 cameras: ~6-8 FPS/stream (YOLOv8n INT8)

42

รูปที่ 42: ขั้นตอน deploy บน Jetson Nano: export ONNX, convert TensorRT, run inference และ multi-camera streaming

ขั้นตอน deploy บน Jetson Nano

18. Export model เป็น ONNX ด้วย `imgsz=416` และ `simplify=True`
19. แปลง ONNX เป็น TensorRT engine ด้วย `trtexec` พร้อม `--fp16` หรือ `--int8`
20. โหลด `model.engine` ด้วย Ultralytics YOLO แล้วรัน `predict`
21. สำหรับหลายกล้อง ใช้ source list เช่น `rtsp://cam1` ถึง `rtsp://cam4`

ข้อจำกัด Jetson Nano

จากสไลด์ Jetson Nano อาจได้ประมาณ 20-25 FPS สำหรับกล้องเดียว, 12-15 FPS ต่อ stream สำหรับสองกล้อง และ 6-8 FPS ต่อ stream สำหรับสี่กล้องด้วย YOLOv8n INT8 ดังนั้นถ้าต้องการ 4 กล้องที่ 25 FPS จริง อาจต้องใช้บอร์ดที่แรงกว่า หรือปรับ resolution, batch, model และ pipeline เพิ่มเติม

บทที่ 12: ตารางเปรียบเทียบเทคนิค Optimization

All Techniques Side-by-Side — Expected Results Dashboard

Every optimization we did — measured and compared

Technique	FLOPs Reduction	Latency Improvement	mAP Impact	Effort
Baseline (FP32)	0%	1x	0%	★
FP16 Quantization	0%	~1.5–2x	~0%	★
INT8 (TensorRT)	0%	~2–4x	~1–2%	★★
Structured Pruning	20–40%	~1.2–2x	~2–5%	★★★
Knowledge Distillation	60–75%	~2–3x	~5–8%	★★★
DW-Sep Heads	85–90% (head)	~1.3x	~1–3%	★★
Image Resize ↓	Quadratic	Quadratic	~3–10%	★

The Pareto Frontier Principle

No single optimization wins everywhere. The art of real-world AI engineering is finding the optimal point on the mAP ↔ Speed ↔ Memory frontier, given your specific constraints (cameras, hardware, acceptable accuracy drop).

43

Teepong Panboonyuen, Ph.D. | SAIE 2026 | Super AI Engineer Thailand

รูปที่ 43: Dashboard เปรียบเทียบเทคนิค optimization ทั้งหมดตาม FLOPs, latency, mAP และ effort

เทคนิค	ช่วยลดอะไร	ความเร็วเพิ่ม	mAP ลด	ความยาก	เหมาะกับ
Baseline FP32	ไม่ลด	1x	0%	ง่าย	ใช้เป็นจุดอ้างอิง
FP16 Quantization	ไม่ลด FLOPs แต่ลด precision	1.5-2x	~0%	ง่าย	GPU ที่รองรับ FP16
INT8 TensorRT	ไม่ลด FLOPs เจริญโครงสร้าง แต่คำนวณถูกลง	2-4x	~1-2%	กลาง	Edge deployment / Jetson
Structured Pruning	20-40%	1.2-2x	~2-5%	ยากกว่า	เมื่อต้องลด compute จริง
Knowledge Distillation	60-75% จากการใช้ student เล็ก	2-3x	~5-8%	ยากกว่า	ต้องการโมเดลเล็กแต่เก่ง
DW-Sep Heads	85-90% ใน head	~1.3x	~1-3%	กลาง	ลด head FLOPs โดยเฉพาะ tiny object
Image Resize	ลดแบบ quadratic	เพิ่มแบบ quadratic	~3-10%	ง่าย	เมื่อยอมลด resolution ได้

บทที่ 13: Final Challenge / Case Study

โจทย์: ออกแบบระบบให้รองรับ 4 กล้อง กล้องละ 25 FPS โดยใช้ GPU เดียว และรักษา mAP@50 อย่างน้อย 80% ของ baseline

คำนวณ requirement

$$4 \text{ cameras} \times 25 \text{ FPS} = 100 \text{ FPS total}$$

ถ้า baseline คือ YOLOv8s แล้วต้องรักษา accuracy อย่างน้อย 80% ของ baseline ดังนั้นโมเดลที่เลือกต้องไม่เร็วอย่างเดียว แต่ต้องมี mAP ไม่ตกมากเกินไป

ตัวเลือก	Estimated FPS	ผ่าน 100 FPS?	mAP >= 80%?
YOLOv8s FP32	~120	อาจไม่ผ่านเมื่อคิด overhead / scale	ผ่าน
YOLOv8s FP16	~200	ผ่าน	ผ่าน
YOLOv8n FP16	~300	ผ่าน	ผ่านประมาณ 87%
YOLOv8n FP16 + KD	300+	ผ่าน	ผ่านประมาณ 92%
YOLOv8n INT8 Jetson	~25 ต่อกล้อง	ผ่านตามเป้าหมายกรณี	ผ่านประมาณ 89%

Recommendation

ตัวเลือกที่เหมาะสมที่สุดตามสไลด์คือ YOLOv8n FP16 + Knowledge Distillation เพราะได้ FPS สูง รักษา mAP ใกล้ teacher และเมื่อใช้ batch inference จะมี headroom สูงมาก สำหรับ Jetson Nano ตัวเลือก INT8 เหมาะกับ 1-2 กล้องมากกว่า ถ้าต้องการ 4 กล้องจริงควรใช้บอร์ดที่แรงขึ้น

บทที่ 14: Key Takeaways

Key Takeaways — What You Can Do After Today

 FP16 is Always On Half precision gives ~1.5–2× speedup with near-zero accuracy cost. Always enable it. INT8 + TensorRT for edge deployment.	 Pruning Removes Redundancy BN γ values reveal which channels are idle. Structured pruning removes them entirely — real FLOPs savings. Fine-tune to recover accuracy.
 Multi-Scale Heads Detect Tiny Objects Drone footage needs the P3 (stride 8) head for tiny objects. DW-Sep convolutions reduce head FLOPs by 8–9× without accuracy sacrifice.	 Deploy with Batch + ONNX + TensorRT Batch inference is the practical multi-camera solution. Export ONNX → TensorRT on Jetson for hardware-optimized edge inference.

รูปที่ 46: สรุป takeaways: Profile ก่อน Optimize, Pruning, FP16, Distillation, Multi-Scale Heads และ Deploy ด้วย Batch + ONNX + TensorRT

- Profile First, Optimize Second: วัดก่อนปรับเสมอ
- Structured Pruning ตัด redundancy ออกจริง แต่ต้อง fine-tune
- FP16 เป็น optimization แรกที่ควรลอง เพราะง่ายและ accuracy แทบไม่ลด
- INT8 + TensorRT เหมาะกับ edge deployment
- Knowledge Distillation ช่วยให้โมเดลเล็กฉลาดขึ้น
- P3 head สำคัญมากกับ tiny objects ในภาพโดรน
- Batch inference + ONNX + TensorRT คือเส้นทาง deploy ระบบหลายกล้อง

บทที่ 15: Glossary คำศัพท์สำคัญ

คำศัพท์	คำอธิบายสั้น
Bounding Box	กรอบสี่เหลี่ยมที่ครอบวัตถุในภาพ
Confidence Score	ค่าความมั่นใจของ prediction
IoU	Intersection over Union ใช้วัดความทับซ้อนระหว่าง predicted box กับ ground truth box
AP	Average Precision ของ class หนึ่ง
mAP	ค่าเฉลี่ย AP ของหลาย class
GFLOPs	พันล้าน floating point operations
P50 Latency	ค่ามัธยฐานของ latency
P95 Latency	latency ที่ 95% ของเคสเร็วกว่าหรือเท่าค่านี้นี้ ใช้ดู worst-case
Pruning	การตัดส่วนที่ไม่จำเป็นของโมเดล
Fine-tuning	การ train ต่อหลังปรับโมเดลเพื่อแก้ performance
Quantization	การลด precision ของตัวเลขในโมเดล
Calibration Dataset	ชุดข้อมูลตัวอย่างที่ใช้หาช่วงค่าที่เหมาะสมสำหรับ INT8
Teacher Model	โมเดลใหญ่ที่แม่นยำกว่า ใช้ถ่ายทอดความรู้
Student Model	โมเดลเล็กที่เรียนจาก teacher
FPN	Feature Pyramid Network สำหรับตรวจจับหลาย scale
Depthwise Conv	convolution ที่ทำแยกตาม channel
Pointwise Conv	1x1 convolution ที่ใช้ผสม channel
TensorRT Engine	ไฟล์โมเดลที่ถูก optimize สำหรับ inference บน NVIDIA hardware

บทที่ 16: แบบฝึกหัดท้ายบทพร้อมเฉลย

ชุดที่ 1: ปรนัย 10 ข้อ

1. Metric ใดใช้วัดความแม่นยำหลักของ Object Detection ในสไลด์? เฉลย: mAP@50
2. ถ้า latency เฉลี่ย 20 ms ค่า FPS เท่ากับเท่าไร? เฉลย: 50 FPS
3. Optimization แรกที่สไลด์แนะนำให้ลองคืออะไร? เฉลย: FP16 Quantization
4. Structured Pruning ตัดอะไรออกจากโมเดล? เฉลย: Channel / filter ทั้งชุด
5. ทำไมต้อง fine-tune หลัง pruning? เฉลย: เพื่อกู้ accuracy ที่ตกหลังตัด channel
6. INT8 ต้องการอะไรเพิ่มเติมเมื่อเทียบกับ FP16? เฉลย: Calibration dataset และ runtime เช่น TensorRT/ONNX
7. Teacher -> Student เป็นแนวคิดของเทคนิคใด? เฉลย: Knowledge Distillation
8. P3 stride 8 เหมาะกับวัตถุแบบใด? เฉลย: Tiny objects
9. Batch inference ช่วยเพิ่มอะไรเป็นหลัก? เฉลย: Throughput รวมของระบบ
10. โจทย์ 4 กล้อง x 25 FPS ต้องการ throughput รวมเท่าไร? เฉลย: 100 FPS

ชุดที่ 2: คำถามอธิบายสั้น 10 ข้อ

1. ทำไม mAP@50-95 เข้มงวดกว่า mAP@50? แนวคำตอบ: เพราะเฉลี่ยหลาย IoU threshold จึงลงโทษ box ที่ไม่แม่นยำมากขึ้น
2. ทำไม FLOPs ต่ำอาจไม่ได้แปลว่าเร็วเสมอ? แนวคำตอบ: เพราะ runtime จริงขึ้นกับ hardware, memory, batch size และ kernel optimization
3. Dark Knowledge คืออะไร? แนวคำตอบ: ข้อมูลความคล้ายระหว่าง class ที่อยู่ใน soft labels ของ teacher
4. ทำไม VisDrone ยากสำหรับ object detection? แนวคำตอบ: เพราะ object เล็กมาก มี class imbalance และมุมมองจากโดรน
5. ทำไม P95 latency สำคัญ? แนวคำตอบ: เพราะบอก worst-case lag ที่ผู้ใช้จะเจอ
6. ONNX มีประโยชน์อย่างไร? แนวคำตอบ: เป็นรูปแบบกลางสำหรับย้ายโมเดลไป runtime/hardware อื่น
7. TensorRT ใช้ทำอะไร? แนวคำตอบ: optimize inference บน NVIDIA GPU/Jetson
8. Depthwise-Separable Conv ลด FLOPs ได้อย่างไร? แนวคำตอบ: แยก spatial filtering กับ channel mixing ออกจากกัน
9. Pareto Frontier หมายถึงอะไร? แนวคำตอบ: จุดที่ปรับ metric หนึ่งดีขึ้นไม่ได้โดยไม่ทำให้อีก metric แย่ลง

10. ทำไม batch inference มี trade-off? แนวคำตอบ: throughput เพิ่มขึ้น แต่ latency ต่อ batch อาจเพิ่ม

ชุดที่ 3: โจทย์คำนวณ 5 ข้อ

1. Latency 10 ms เท้ากับ FPS? เฉลย: $FPS = 1000/10 = 100$ FPS
2. 4 กล้อง กล้องละ 25 FPS ต้องการ throughput เท่าไร? เฉลย: $4 \times 25 = 100$ img/s
3. โมเดลมี $mAP@50 = 0.45$ และ GFLOPs = 28.6 Efficiency เท่าไร? เฉลย: $0.45/28.6 \approx 0.0157$
4. FP32 model 100 MB ถ้าใช้ FP16 ขนาดประมาณเท่าไร? เฉลย: ประมาณ 50 MB
5. FP32 model 100 MB ถ้าใช้ INT8 ขนาดประมาณเท่าไร? เฉลย: ประมาณ 25 MB

ชุดที่ 4: คำถามวิเคราะห์ Case Study 3 ข้อ

1. ถ้าต้อง deploy บน Jetson Nano 4 กล้องที่ 25 FPS ควรระวังอะไร? แนวคำตอบ: Jetson Nano อาจไม่พอสำหรับ 4 กล้องจริง ต้องพิจารณา INT8, ลด resolution, batch/pipeline และอาจใช้บอร์ดที่แรงกว่า
2. ถ้า mAP ลดมากหลัง pruning ควรทำอย่างไร? แนวคำตอบ: ลด pruning ratio, fine-tune นานขึ้น, ใช้ sparsity training หรือเลือก prune เฉพาะ layer ที่ปลอดภัย
3. ถ้าระบบต้องการ accuracy สูงมากกว่า speed ควรเลือกโมเดลแบบใด? แนวคำตอบ: โมเดลใหญ่ขึ้น เช่น YOLOv8s/m หรือใช้ teacher model แต่ต้องตรวจว่าผ่าน FPS requirement หรือไม่

บทที่ 17: สรุปสุดท้ายแบบอ่านก่อนสอบ

สิ่งที่ต้องจำ

- ระบบจริงต้องดูทั้ง mAP, FLOPs, latency, FPS และ memory
- 4 cameras x 25 FPS = 100 FPS รวม
- FP16 เร็วขึ้นง่าย accuracy แทบไม่ลด
- INT8 เหมาะกับ edge แต่ต้อง calibrate
- Pruning ต้องเป็น structured จึงลด FLOPs ได้จริง
- Distillation ทำให้ student เล็กแต่เก่งขึ้น
- P3 stride 8 สำคัญกับ tiny objects

สูตรที่ต้องจำ

$$mAP = (1 / |C|) \times \sum AP_c$$

$$\text{FPS} = 1000 / \text{mean_latency_ms}$$

$$\text{Efficiency} = \text{mAP@50} / \text{GFLOPs}$$

$$\text{FLOPs Conv2D} = 2 \times C_{\text{in}} \times k^2 \times C_{\text{out}} \times H \times W$$

จุดที่มักสับสน

- Latency คือเวลาต่อภาพ แต่ throughput คือปริมาณภาพรวมต่อวินาที
- mAP@50 ง่ายกว่า mAP@50-95
- FLOPs ไม่ใช่ latency แต่เป็นตัวประมาณต้นทุน compute
- FP16 กับ INT8 เป็น quantization แต่ INT8 มี calibration และ accuracy drop มากกว่า
- Batch inference ช่วย throughput แต่ต้องระวัง latency

ประโยคจำง่ายก่อนสอบ

Profile ก่อน Optimize, FP16 ก่อน INT8, Structured Pruning ก่อนห้วงลด FLOPs, Distill เพื่อให้โมเดลเล็กเก่งขึ้น, และ Deploy
จริงต้องคิดทั้ง Batch + ONNX + TensorRT

แหล่งอ้างอิงจากไฟล์ที่ใช้จัดทำ

- SAIE2026_ObjectDetection_PanboonyuenLecture(1).pdf - AI in the Real World: Trade-offs Behind Fast & Scalable Object Detection
- ข้อความ Prompt ที่แนบมา - ใช้เป็นข้อกำหนดรูปแบบเอกสารภาษาไทย